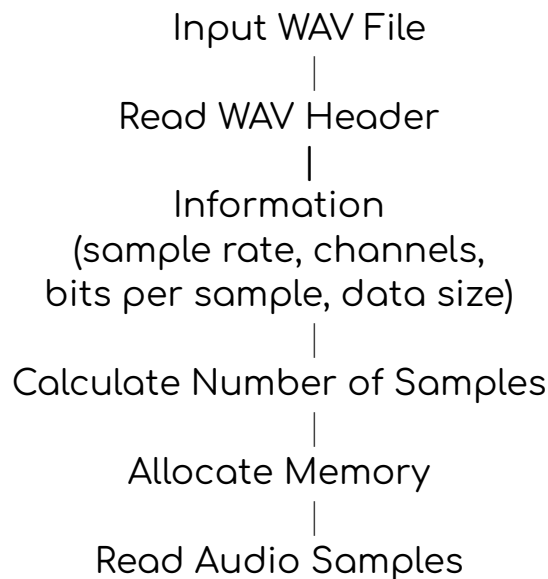


# PROJECT -

Implementing 3 DSP techniques

- Moving Average Filter
- Wiener Filter
- Spectral subtraction

STEP 1 - READING A WAV FILE :

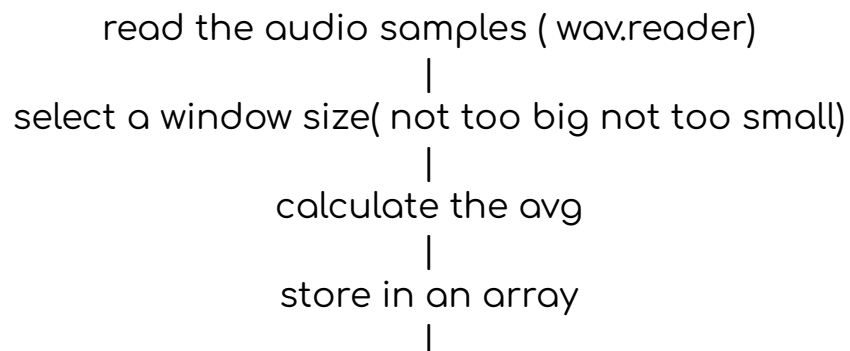


## 1) MOVING AVERAGE FILTER -

How it works -

The Moving Average Filter considers a small window of neighboring samples, and for each sample -

1. A window size is selected
2. Then compute the avg
3. replace the current sample with the avg



write the filtered audio

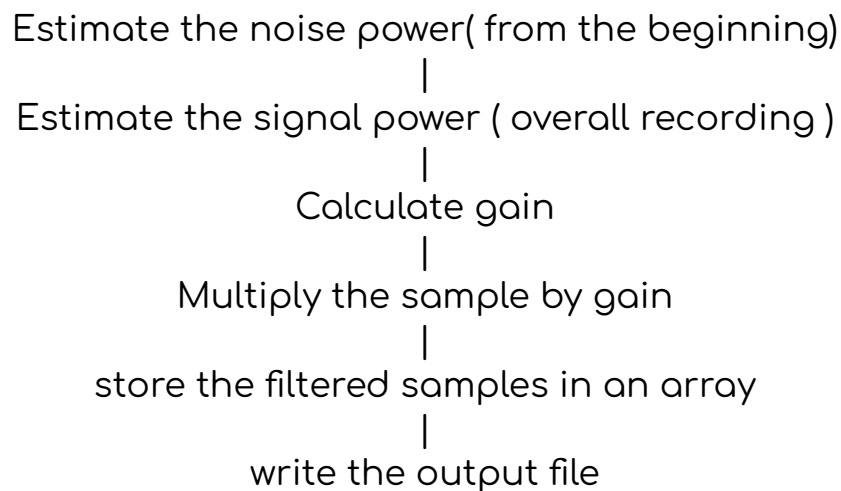
Problems-

- Smoothens the entire audio without separating speech from noise
- selecting the correct window size so that the audio is not affected a lot ( speech was muffled for very high window sizes)

## 2. WIENER FILTER -

How it works -

Initial implementation - I assumed that the beginning of the audio contained only background noise. So I used that portion to calculate the avg noise power

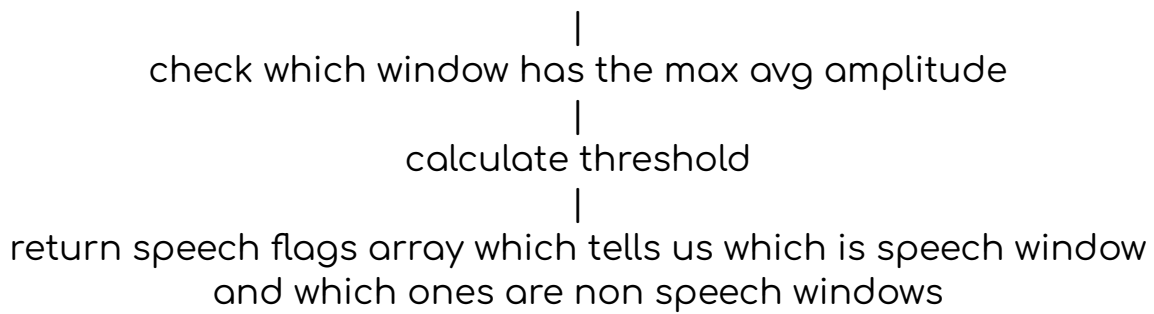


Improved implementation-

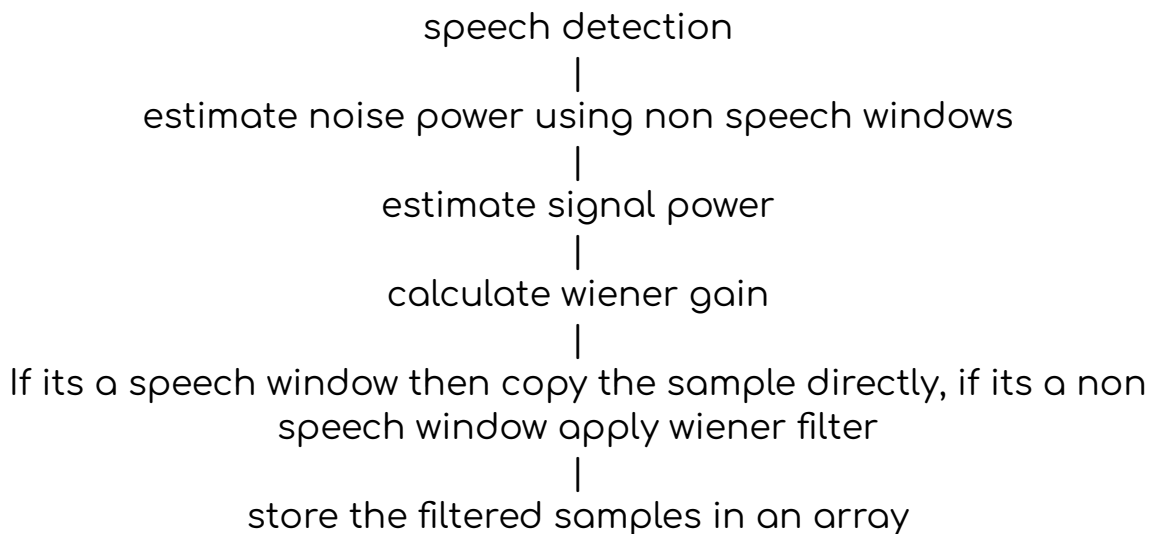
Instead of assuming where the noise exists in the recording, I first identify where the audio contains speech and where it contains only background noise.

Speech detection -





After speech detection the improved wiener filter -

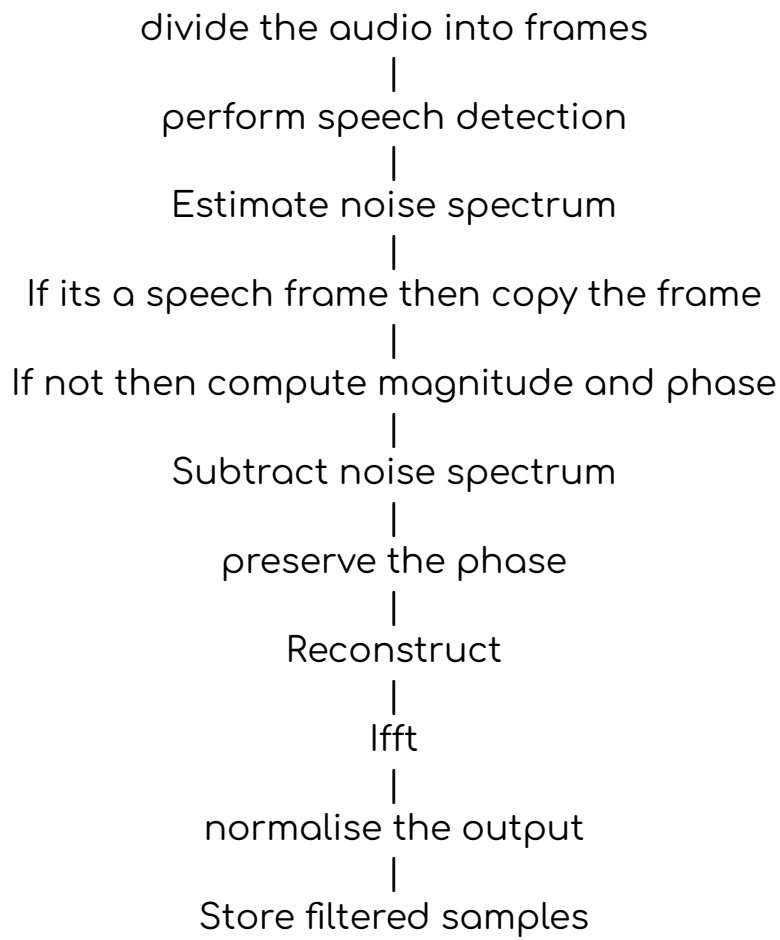


- $\text{Gain} = \text{signal power} / \text{signal power} + \text{noise power}.$

### 3. SPECTRAL SUBTRACTION -

HOW IT WORKS - the audio is first divided into frames. Each frame is converted from time domain to frequency domain using FFT. Since background noise usually occupies different frequencies as compared to speech, the frequency domain allows the noise to be filtered better, while preserving the speech components.

The scope of the project was not to implement an FFT algorithm from scratch, that is why I have made use of KissFFT library which is an open source C library.



A frame size of 256 was chosen because it looks like a good balance between a very high number and very low number which could disrupt the process.

One problem I overcame -

Initially I did not normalize the output, which led to the filtered audio looking and sounding extremely distorted with aptitudes splurging.

The reason to normalize is because the output that is produced by IFFT is scaled by frame size, which results in amplitudes that are larger than original values. To restore the correct amplitude each output sample is divided by the frame size.

## FINALLY (FOR AUDIO 2)



